Build & Publish an Android Game with AI

A hands-on vibe-coding workshop guide

Prompt Guide • Step-by-Step Guide

Build Your Own Game

The game shown in this guide (Space Escape Runner) is only a reference project used during the Live Workshop to demonstrate the complete app development workflow.

You are not required to build the same game.

In fact, we encourage you to be creative and build any Android game or app of your choice. You can use this guide as a reference to understand how to structure prompts, build features incrementally, and work with AI throughout the development process.

Examples of projects you can build include:

- 🎮 Flappy Bird
- 🏓 Pong
- 🐍 Snake Game
- 🧱 Brick Breaker
- 🚀 Endless Runner
- 🧩 Memory Game
- ❌ Tic Tac Toe
- 🚀 Anything you like to build

The most important goal is to learn the workflow, not recreate the exact project.

Build something you're excited about, experiment with AI, and make it your own! 🚀

1. Prompt Guide

This section is your prompt-by-prompt script for building **Space Escape Runner** with AI. Copy each prompt into your AI coding assistant (Cursor, or VS Code with Claude) exactly as written, in order. Each phase builds on the previous one, so don't skip ahead. Wait for one prompt to finish and test the result before moving to the next.

How to use it: run one prompt → read the AI's explanation → save the file → check it on your phone in Expo Go → then run the next prompt. This incremental, one-concept-at-a-time approach is what keeps a beginner from getting lost.

Phase 1 — Project Setup & First Screen

Prompt 1a — Create the project

```
I am a complete beginner in React Native and Expo.
Help me create a new Expo project called SpaceEscapeRunner.
Give me:
1. Installation prerequisites
2. Commands to create the project
3. Commands to run the project
4. Explain what each command does
5. How to test the app using Expo Go
Do not generate game code yet.
```

Prompt 1b — First game screen

```
The Expo project is running successfully.
Generate a simple game screen with:
- Title: Space Escape Runner
- Current Score
- Start Game Button
Use a clean modern UI.
Explain every line of code for a beginner.
```

Phase 2 — The Spaceship & Movement

Prompt 2a — Add the spaceship

```
Now add a spaceship at the bottom of the screen.
Requirements:
- Use React Native Views only.
- Do not use images.
- Represent the spaceship using shapes and colors.
- Position it at the bottom center.
Explain all styling and positioning logic.
```

Prompt 2b — Left / right controls

```
Add left and right movement controls.
Requirements:
- Add two buttons: Move Left and Move Right.
- Spaceship should move horizontally.
- Prevent moving outside screen boundaries.
Explain state management step by step.
```

Phase 3 — Asteroids, Game Loop & Collisions

Prompt 3a — Falling asteroids

```
Add falling asteroids.
Requirements:
- Generate asteroid at a random x position.
- Asteroid falls continuously from the top.
- When asteroid reaches the bottom:
    - Create a new asteroid.
    - Increase the score.
Keep the implementation beginner friendly.
Explain the game loop logic.
```

Prompt 3b — Collision detection

```
Add collision detection.
Requirements:
- Detect collision between asteroid and spaceship.
- Show a Game Over message.
- Stop the game loop.
- Show the final score.
Explain the collision calculation step by step.
```

Phase 4 — Restart & High Score

Prompt 4a — Restart functionality

```
Add Restart Game functionality.
Requirements:
- Reset score
- Reset asteroid position
- Reset spaceship position
- Start the game again
Explain how state reset works.
```

Prompt 4b — High score with AsyncStorage

```
Add local high score storage using AsyncStorage.
Requirements:
- Save the highest score
- Load the high score when the app starts
- Display the high score on screen
Explain AsyncStorage in simple terms.
```

Phase 5 — Polish the UI

Prompt 5 — Visual polish

```
Improve the UI. Add:
- Gradient background
- Better spaceship design
- Better asteroid design
- Smooth animations
- Modern game UI
Keep the code simple and Expo compatible.
```

Phase 6 — Build with EAS

Prompt 6 — Generate the Android build

```
The game is complete. Guide me step by step to:
1. Create an Expo account
2. Configure EAS
3. Generate an Android APK
4. Generate an Android AAB
5. Download the build
Explain each command and the expected output.
```

Phase 7 — Publish to Google Play Console

This is the longest prompt. It asks the AI to walk you through the entire Play Console publishing journey. Paste it as a single prompt so the AI produces one connected, end-to-end guide.

Prompt 7 — Full Play Console walkthrough

```
The Android AAB build is ready. Guide me step by step to publish
Space Escape Runner on the Google Play Console using a developer
account. Explain the complete process, including:

1. Google Play Developer Account Setup
   - How to create a Google Play Developer account
   - The one-time registration fee and why it's required
   - Identity verification requirements and expected timeline
2. Creating the App in Play Console
   - How to create a new app entry
   - App name, default language, app vs game, free/paid status
3. Store Listing Setup
   - Required assets: app icon, feature graphic, screenshots
     (with exact dimensions)
   - Writing the short description and full description
   - Selecting app category and adding contact details
4. Content and Compliance Forms
   - Privacy policy requirement (and how to create a simple one)
   - Content rating questionnaire
   - Target audience and age settings
   - Data safety form
   - Ads declaration and government-app declaration
5. Uploading the Build
   - How to upload the AAB file
   - Internal vs Closed vs Open vs Production tracks
   - Why Google requires closed testing before production
     for new personal accounts
6. App Signing
   - What Google Play App Signing is
   - How the EAS-generated key relates to Google's signing key
   - Why I should not lose my upload keystore
7. Release and Review
   - How to create a production release
   - Submitting for review and expected review timeline
   - Common reasons apps get rejected and how to avoid them

For each step, explain what I'm doing, why it's required, and
what I should expect to see on screen. Assume I am publishing my
first app and have never used the Play Console before. Keep it
beginner friendly.
```

2. Step-by-Step Guide

The Prompt Guide tells you *what to type*. This section tells you *what to do around it* — the setup before you start, the workflow while you build, and the deployment at the end. Follow it top to bottom.

Before You Start: Prerequisites Checklist

Set these up in advance. The account verifications take time, so don't leave them for the last minute.

- Node.js (LTS version) installed on your computer — this runs all the tools.
- A code editor: Cursor, or VS Code with an AI assistant (Claude).
- The Expo Go app installed on your Android phone (from the Play Store) — this is how you test instantly.
- A free Expo (EAS) account created in advance — needed for one-click cloud builds.
- A Google Play Developer account **(optional)** — only if you plan to actually publish. It has a one-time Google fee and identity verification that can take a few days, so set it up early.

The Tech Stack

- **Editor:** Cursor or VS Code with Claude.
- **Framework:** React Native (Expo). Expo is ideal for vibe-coding because it handles the heavy lifting of Android builds and lets you test instantly on a real phone via Expo Go.
- **Build service:** Expo Application Services (EAS) for cloud builds — no Android Studio needed.
- **Assets:** For this game we use shapes and colors only (no image files), which keeps things simple. AI image tools are optional for icons and graphics later.

The Build Roadmap

Here is the full journey at a glance. Each phase maps to the prompts in Section 1.

Phase	Activity	Vibe-Coding Technique
I: Brainstorm	Define the game mechanic (a simple 2D runner / dodge game).	Use a system prompt to describe the vibe and rules to the AI.
II: Scaffolding	Initialize the Expo project structure and game screen.	Command the AI to generate App.js and the game-loop skeleton.
III: Development	Build player movement, falling asteroids, scoring, collisions.	Natural-language refinement — describe the feel and iterate.
IV: Polish	Add gradient background, better visuals, animations, high-score storage.	Ask the AI to integrate assets and manage state.
V: Deployment	Generate the .aab build and submit to Google Play.	Use Expo Application Services (EAS) for cloud builds.

The Core Working Loop

For every phase, repeat this five-step loop. This is the single most important habit — build in small pieces and confirm each one works before adding the next.

1. Run the prompt for the current phase (from Section 1).
2. Read the AI's explanation. Don't just copy the code — understand what each part does.
3. Save the file(s) the AI gives you.
4. **Test on your phone in Expo Go.** This is your checkpoint — confirm the new feature works before moving on.
5. Only when it works, move to the next prompt. If something breaks, describe the problem back to the AI and iterate.

Phase-by-Phase Walkthrough

Phase 1 — Setup & First Screen

Run Prompt 1a to install prerequisites and create the project, then scan the QR code with Expo Go to see a blank app running on your phone. Run Prompt 1b to get your first screen: the title, a score display, and a Start button. Checkpoint: the screen appears on your phone.

Phase 2 — Spaceship & Movement

Prompt 2a draws the spaceship from plain Views (shapes and colors). Prompt 2b adds Move Left / Move Right buttons and keeps the ship inside the screen. Checkpoint: the ship moves and stops at both edges.

Phase 3 — Asteroids, Loop & Collisions

Prompt 3a adds asteroids that fall from random positions and increase your score, driven by a game loop. Prompt 3b detects when an asteroid hits the ship, shows Game Over, and stops the loop. Checkpoint: score rises as asteroids pass, and a hit ends the game.

Phase 4 — Restart & High Score

Prompt 4a adds a Restart button that resets score, ship, and asteroids. Prompt 4b saves your best score with AsyncStorage so it survives closing the app. Checkpoint: restart works, and your high score is remembered after you reopen the app.

Phase 5 — Polish

Prompt 5 adds a gradient background, nicer ship and asteroid designs, and smooth animations. Checkpoint: the game looks finished and still runs smoothly on your phone.

Phase 6 — Build the App

Prompt 6 walks you through creating your Expo account, configuring EAS, and generating the Android APK and AAB in the cloud. The APK is for quick install/testing on a phone; the AAB is the file Google Play requires. Checkpoint: you have downloaded a .aab file.

Phase 7 — Publish

Prompt 7 walks you through the full Play Console process. Because Play review times vary from hours to days, finishing the session usually means successfully submitting the build for review and knowing how to manage the Console — not necessarily seeing it go live the same day.

Key Milestones to Aim For

- **The power of iteration:** you learn to prompt-engineer complex logic one step at a time.
- **A working Minimum Viable Game:** a complete, playable game with scoring, game-over, restart, and a saved high score.
- **Understanding the Android ecosystem:** icons, screenshots, privacy policy, and the Play Store publishing flow.

Pro Tips

- Start simple and get the full pipeline working before adding extras — a running build is worth more than an unfinished feature.
- Set up your Expo and (optional) Play Developer accounts before the session; verification takes time.
- If a prompt's output breaks, paste the error back to the AI and describe the feel you want — iteration is the whole point of vibe-coding.

Project Repository

The complete source code for the reference project is available on GitHub:

https://github.com/SaumyaPatil/SpaceEscapeRunner.git

Clone it to explore the finished code, or use it as a reference while building your own game:

bash: git clone https://github.com/SaumyaPatil/SpaceEscapeRunner.git